Ders 7 Genişletilmiş Notu: Bağlı Listeye Giriş: Düğüm Oluşturma, Ekleme ve Yazdırma

Ders 7 Genişletilmiş Notu: Bağlı Listeye Giriş: Düğüm Oluşturma, Ekleme ve Yazdırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dizilerden Bağlı Listeye
- 2. Bağlı Liste Yapısı
- 3. Avantajlar ve Dezavantajlar
- 4. Tek Bağlı ve Çift Bağlı Liste
- 5. C ile Düğüm Tanımı ve Düğüm Oluşturma
- 6. Yeni Eleman Nereye Eklenir?
- 7. Başa Ekleme
- 8. Sona Ekleme
- 9. `createNode` Nereden Çağrılmalı?
- 10. Listeyi Yazdırma
- 11. İkili Ağacı Düz Dizide Tutmanın Sorunu
- 12. Araya Ekleme: Sıralı Liste
- 13. Sıradaki Konu: Düğüm Silme

1. Dizilerden Bağlı Listeye

İlk haftalarda algoritma tasarlarken ve analiz ederken nelere dikkat edileceğini gördük. Örneklerin hepsi arama ve sıralama algoritmalarıydı ve hepsi dizi kullanılarak gerçekleştirildi. Hoca yeni konuya bu noktadan girdi: Dizi bildiğimiz en temel veri yapısıdır, ama her ihtiyacı karşılamaz.

1.1 Dizi: En Temel Veri Yapısı

Aynı amaç için kullanılan birden fazla bilginiz olduğunda bunları ayrı ayrı değişkenlerde tutmazsınız. Bellekte ardışık yerleşmiş bir bölgede, yani bir dizide tutarsınız. Her elemanın kendine ait bir yeri olduğu için gerektiğinde adresini kullanarak erişirsiniz. Örneğin "A dizisinin 3. gözüne eriş" dersiniz ve o elemana doğrudan ulaşırsınız.

1.2 Statik Dizi ve Sınırı

Kod yazarken ilk öğretilen yol statik yer ayırmaktır. Örneğin `int A[50]` ile 50 elemanlık yer tutulur. Dışarıdan okunan n değeri kaç ise dizinin o kadarlık kısmı kullanılır. Bunun bir zorunluluğu vardır: Kod yazarken n'in bu sınırı aşmayacağını garanti etmeniz gerekir.

> **Ders sonrası not:** Hoca n'in mutlaka 50'nin altında olması gerektiğini söyledi; altyazıda kullanılabilen gözler de "sıfırdan dokuzuncu göze kadar" diye geçiyor. Doğrusu: `int A[50]` dizisinin gözleri 0'dan 49'a kadardır. Bu nedenle $n \leq 50$ yeterlidir.

1.3 Dinamik Bellek Ayırma

Sonraki adım programlama derslerinde görülen dinamik bellek ayırmadır (dynamic memory allocation). Dizinin uzunluğu bazen 50, bazen 1000 olabilir. Olabilecek en büyük uzunluğa göre gereksiz yere statik yer tutmak yerine n dışarıdan alınır ve o çalıştırmada kaç gerekiyorsa o kadar yer ayrılır. n'i okuduğunuzu varsayarsak `n * sizeof(int)` kadar yer istemek, belleğin heap bölgesinde n elemanı saklayacak kadar **ardışık** bir bölge ayırmak demektir.

1.4 Dinamik Dizinin de Yetmediği Durumlar

Bazı durumlarda bu da yetmez. Hoca iki durum saydı.

Birincisi, baştan kaç elemanınız olacağını hiç bilmiyor olabilirsiniz. Yine birbiriyle ilişkili, bir arada bulunması gereken bir grup bilgi vardır, yani dizi mantığı sürer. Ama yeni elemanlar geldikçe onlar için yer tutmanız gerekir. Aksi hâlde yine statik durumdaki gibi çok büyük bir yer tutup elemanları geldikçe yerleştirirsiniz.

İkincisi, ekleme ve silme çok sık yapılıyor olabilir. Bir eleman gelir, işi biter, gider. Burada hoca silmenin ne anlama geldiğini hatırlattı: Silme dediğimiz şey fiziksel bir silme değildir. "O yeri bıraktım, başkası kullansın" demektir. Bu gibi durumlarda elemanlar için tek tek yer ayırmak gerekebilir.

1.5 En İyi Örnek: İkili Arama Ağacı

Hocaya göre bu ihtiyacın en iyi örneği, Ayrık Matematik dersinde görülen ağaç yapısıdır. O derste yapılar anlatılmıştı, algoritmalarını bu derste göreceğiz. Hoca basit bir ikili arama ağacı (binary search tree) çizdi. Dışarıdan yeni bir eleman gelir, ağaçta böyle bir eleman yoktur, geldiğinde uygun yere eklenir. Sonra bir tane daha gelir, örneğin 4 gelir, o da kendi yerine eklenir.

Başlangıçta bu elemanların hiçbiri yoktu. Gerçek dünyada her şey baştan bir seferde belli değildir, geldikçe işlem yapmanız gerekir. Bunun tersi de olur: Örneğin 6'nın işi biter ve silinmesi gerekir, yani ağaçtan çıkarılır. Bazı algoritmalarda böyle eklemeler ve çıkarmalar vardır ve başlangıçta bunların ne kadar olacağını bilmezsiniz. Bu tür işler için kullanılan veri yapısı bağlı listedir (linked list).

2. Bağlı Liste Yapısı

2.1 Düğüm: Değer ve Adres

Bağlı listede her seferinde tek bir eleman için yer tutulur. Elemanlar geldikçe eklendiği için bellekte arka arkaya boş yer bulunacağının garantisi yoktur, dolayısıyla arka arkaya duramazlar. Oysa birbirlerine bağlı kalmaları, kaybolmamaları gerekir. Bu yüzden her eleman iki kısımdan oluşur:

- **Değer (value):** Elemanın taşıdığı bilgi. Bu kısım gerçekte birden fazla bilgiden oluşabilir: ad, soyad, adres, bir yıl, bir numara gibi. Hoca basitlik için ve vakit kaybetmemek için tek bir değer kullandı.
- **Adres:** Bir sonraki elemanın yerini gösterir. Bu kısım mutlaka vardır. Hocanın deyişiyle, öbür türlü elemanlar birbirlerine tutunamaz, kaybolur.

Yani elemanlar birbirine göstericilerle (pointer) bağlıdır. Her eleman kendisinden sonra gelen elemanın adresini gösterir. Yeni bir eleman geldiğinde ihtiyacınıza ve algoritmanızın tasarımına göre uygun bir yere bağlanır. Herkes birbirini gösterdiği için bir arada duran, birbirine bağlı bir bilgi grubu elde edilir.

2.2 Adreslerle Örnek

Hoca adres alanının gerçekten bellek adresi tuttuğunu sayılarla gösterdi. İlk elemanın bellekte 1000 numaralı adreste olduğunu varsayalım.

- Yeni bir eleman gelir ve yer ayrılınca 1400 numaralı adrese yerleşir. O zaman 1000'deki elemanın adres alanı 1400 olur.

- Bir eleman daha gelir ve 1500 numaralı adrese yerleşir. O zaman 1400'deki elemanın adres alanı 1500 olur.
- Bundan sonra kimse yoksa 1500'deki elemanın adres alanı `NULL` değerini gösterir.

```
head --> [1000]        [1400: 9]        [1500]
         adres: 1400   adres: 1500      adres: NULL
```

Hoca 1400'deki elemanın değerini 9 olarak verdi.

2.3 `head`

Elemanlara erişebilmek için bir başlangıç noktası gerekir. Bu yüzden ilk elemanın adresini gösteren bir değişken daha olması lazımdır. Hoca buna `head` dedi. `head` yalnız başlangıç adresini gösterir, bir değer tutmaz. Yalnız işe nereden başlayacağımızı, yani elemanların ilkinin nerede durduğunu söyler.

2.4 Doğrusal Ama Sıralı Erişimli

Bağlı liste, dizi gibi doğrusal (lineer) bir veri yapısıdır. Elemanlar doğrusal olarak birbirine bağlanır. Fakat dizide olduğu gibi bir elemana birdenbire erişmek yoktur. Dizide "birinci göze git" dediğinizde 9 hemen gelirdi. Burada ise **kasetçalar mantığı** vardır: Sırayla ilerlemek zorundasınız.

9'a gitmek için önce `head`'den başlarsınız. `head` sizi ilk elemana götürür. Oradan ikinci elemanın 1400 adresinde olduğunu öğrenirsiniz. 1400 adresine gider ve 9'u orada bulursunuz. Zıplayarak birdenbire 22'ye erişme şansınız yoktur. Bir elemana erişmek istediğinizde en baştan başlayıp önceki bütün elemanların üzerine uğrayarak gitmek zorundasınız.

3. Avantajlar ve Dezavantajlar

Hoca bu listeyi öğrencilerle birlikte çıkardı.

3.1 Avantajlar

- **Boyut sabit değildir.** Eleman ekledikçe dinamik olarak büyür.
- **Ekleme ve silme kolaydır.** Yeni bir eleman eklemek ya da aradan bir eleman silmek kolaydır.

Soru: Avantajlara, elemanların bellekte ardışık bulunmak zorunda olmamasını da ekleyebilir miyiz?

Cevap: Evet, bu da bir avantajdır, ama hem avantaj hem dezavantaj tarafı vardır. Dinamik ayırmada yeteri kadar ardışık yer yoksa sıkıntı olurdu. Bellekte kopuk kopuk boş bölgeler olabilir: şurası boş, burası dolu, öbür taraf yine boş. Dinamik ayırma ise ardışık bir bütün yer ister. Bağlı listede böyle bir sıkıntı yaşanmaz.

3.2 Dezavantajlar

Rastgele erişim (random access) yoktur. Hocaya göre en büyük sıkıntı budur, çünkü aramaları etkiler. Hoca bunu ikili aramayla (binary search) sordu.

Soru (hocadan): Elemanlarınız sıralı olsun. Bu yapıda $\log n$ karmaşıklıklı bir ikili aramanız olabilir mi?

Cevap (öğrenciden): Olamaz, çünkü doğrudan istediğimiz yere geçemiyoruz.

Hoca bunu onayladı. $n/2$. göze gitmek için $n/2$ adım ilerlemeniz gerekir. O göze ulaşmak için aradaki elemanların üzerinden geçmek zorundasınız. Bu yüzden böyle bir yapıda $\log n$ karmaşıklıklı bir arama olamaz.

Sonraki dezavantaj sorulduğunda iki yanıt geldi ve hoca ikisini de eledi.

Soru: Uzunluğunu bilememek bir dezavantaj olabilir mi?

Cevap: Hayır, o bir avantajdır.

Soru: Dikkatli olmazsak erişimi kaybedebiliriz. Bu, bizden kaynaklı bir dezavantaj sayılır mı?

Cevap: O, kullanandan kaynaklıdır. Erişimi kaybetmemek gerekir. Burada kullanıcı hatası değil, yapının kendisiyle ilgili dezavantajlar konuşuluyor.

Ek yer ihtiyacı. Bir öğrenci adres alanlarının eklenmesini önerdi, hoca da onayladı. Bütün elemanlar için ek bir gösterici değişken gerekir. 1000 elemanınız varsa, bir sonraki elemanın adresini tutan 1000 değişkene daha ihtiyacınız vardır. Hoca bunu bir yer kaybı olarak nitelendirdi.

Önbellek (cache) dostu değildir. Hoca bunu tahmin etmesi güç olacağı için kendisi söyledi. Sık kullanılan elemanlar önbellekte tutulur. Bir diziyi alıp onun üzerinde önbellek kullanabilirsiniz. Bağlı liste elemanları ise arka arkaya durmaz, belleğin farklı bölgelerinde ayrı ayrı durur. Onları göstericiyle gösteriyor olsanız bile bu yapı önbellek dostu (cache friendly) değildir.

3.3 Ne Zaman Bağlı Liste?

Sık eleman ekleme ve silme gerekiyorsa, eleman sayısı sabit değilse ve ne zaman yeni eleman geleceği belli değilse, bağlı liste diziden çok daha avantajlıdır. En çok da ağaçlar (trees) ve graflar (graphs) bağlı liste yapısını kullanır. Bu derste de rahatça kullanılacaktır.

4. Tek Bağlı ve Çift Bağlı Liste

Şimdiye kadar anlatılan, her elemanın kendisinden sonra geleni gösterdiği yapı **tek bağlı liste (single linked list)** yapısıdır. Tek bir bağ vardır. Bir de **çift bağlı liste (double linked list)** vardır: Her eleman hem kendisinden önce geleni hem kendisinden sonra geleni gösterir. Hoca önce tek bağlı listeyi anlatacağını, çift bağlı listeye ondan sonra geçeceğini söyledi.

Tek bağlı listede her zaman bir `head` vardır ve ilk elemanın adresini gösterir. Bunlar gerçekten adres tutar. Örneğin ilk eleman 1000 numaralı adresteyse `head` 1000'i gösterir. İkinci eleman 1500'deyse ilk eleman 1500'ü gösterir. Üçüncü eleman 2000'deyse ikinci eleman 2000'i gösterir. Herkes kendisinden sonra gelen elemanın bellekteki yerini gösterir.

Bağlı liste, belleği ardışık kullanan normal bir dizi olmadığı için ekleme ve silme işlemleri de farklıdır. Hoca bunları adım adım göstereceğini söyledi. Bir eleman geldiğinde yapılacak ilk iş o eleman için yer tutmaktır.

5. C ile Düğüm Tanımı ve Düğüm Oluşturma

5.1 Neden C ve Dilin Yeri

Bağlı liste yapısı başka dillerde de, örneğin Java'da da kullanılır. Bu derste programlama dili olarak C kullanılır. Hoca bunun sebebini şöyle açıkladı: Hem veri yapıları hem algoritma analizi dersinde hedef, programlama dillerini daha iyi kullanmanızı sağlamak değildir. Hedef, iyi algoritma tasarımını görmeniz, yeri ve zamanı daha iyi kullanan algoritmalar tasarlayabilmenizdir.

Bu yüzden dil çok temel bir şekilde kullanılır. Burada `if`, `for`, `while` önemlidir. Kütüphaneler ve benzeri ayrıntılar bu dersin konusu değildir. Dillerle ilgili ayrıntılar nesneye dayalı programlama ve programlama dilleri gibi derslerde gösterilir.

Hoca C'de bellek ayırırken standart kütüphanenin (standard library) kullanıldığını, bunun genelde yeterli olduğunu hatırlattı.

5.2 Düğüm Yapısı ve Global `head`

Düğümde yalnız değer değil, bir sonraki elemanın adresi de saklanmak zorundadır. Bu yüzden `struct` kullanılır. Hoca basitlik için değeri tek bir tam sayı olarak aldı. Öğrenciler bu yapıyı Yapısal Programlama dersinde görmüştü.

```c
#include <stdlib.h>

typedef struct Node {
    int value;
```

```
    struct Node *next;
} NODE;

NODE *head;
```

Buradaki mantık özyinelemelidir (recursive). Bir sonraki eleman da aynı tipte, yani `struct Node` tipinde bir elemandır. O elemanın adresini gösterecek gösterici değişkenin tipi de bu nedenle `struct Node *` olmak zorundadır.

İlk elemanın adresi yukarıda görüldüğü gibi global bir `head` değişkeninde tutulur. Oysa öğrenciler global değişken kullanmamaları konusunda ciddi biçimde uyarılır. Hoca nedenlerini saydı:

- Fonksiyondan fonksiyona geçerken global değişkenin durumunu takip etmek zordur.
- Değişken bütün fonksiyonlarda kullanılmıyorsa, örneğin yalnız iki fonksiyonda kullanılıyorsa, hangi fonksiyonların kullandığını izlemek zorlaşır.
- Değişken gereksiz yere tutulmuş olur.

Ama bütün fonksiyonlarınız tarafından kullanılacak bir değişkeniniz varsa onu global tanımlayabilirsiniz. `head` böyle bir değişkendir, herkes onu kullanacaktır. Bu yüzden burada global tanımlamanın sakıncası yoktur.

5.3 `createNode`

Yeni gelen bir eleman için ilk iş yer tutmaktır. Yeni elemanın adresini geri döndürebilmek için dönüş tipi `NODE *` olan bir fonksiyon yazılır.

```
NODE *createNode() {
    NODE *p;
    p = (NODE *) malloc(sizeof(NODE));
    if (p == NULL) {
        printf("Allocation error\n");
        exit(0);
    }
    return p;
}
```

`p` göstericisi, ayrılan yerin başlangıcını gösterecektir. Dönüş tipi düğüm göstericisi olduğu için `malloc`'un sonucu `(NODE *)` ile tip dönüşümüne (casting) uğratılır. Tutulacak yer `sizeof(NODE)` kadardır. Yer bulunduysa ayrılan yerin başlangıç adresi geri döndürülür.

> **Hocanın vurgusu:** `malloc`'tan sonra `NULL` kontrolünü her zaman yapmalısınız. Genellikle yer sorunu yaşamazsınız. Fakat uzaya füze atıyorsanız ve o noktada milyonda bir ihtimalle yer problemi olursa füzeniz yere çakılır. "Aman, nasılsa yer vardır" diye düşünmemeniz gerekir.

Yer yoksa hoca örnekte programdan çıktı. Gerçek bir programda, algoritmanızın ihtiyacına göre ne yapılması gerekiyorsa o yapılır.

> **Ders sonrası not:** Kodda `printf` kullanıldığı için `#include <stdio.h>` de gerekir. Ekrandaki kodda yalnız `stdlib.h` var. Bu hâliyle `gcc -Wall` derlemesi `implicit declaration of function 'printf'` hatası verdi. Ayrıca `exit(0)` başarılı çıkış kodu döndürür. Bellek ayırma başarısız olduğunda test programı `Allocation error` yazıp 0 koduyla çıktı. Hata durumunda `exit(EXIT_FAILURE)` yazmak doğrudur.

5.4 Ana Programda Kullanım

Hoca bu fonksiyonun bir ana programda nasıl kullanılacağını gösterdi. İşe başlarken `head` `NULL` değerindedir. Önce bir düğüm için yer ayrılır, sonra örneğin 8 sayısı listeye eklenir.

```
int main()
{
    NODE *p;
    head = NULL;
    p = createNode();
```

```
    insertAtFront(p, 8);
    ...
}
```

Bu parça yalnız çağrı biçimini gösterir. `insertAtFront` sonraki bölümde yazılacaktır.

6. Yeni Eleman Nereye Eklenir?

Hoca eklemeye (insert) en basit durumdan başladı. Hiç eleman yoksa `head` `NULL` gösterir. Yer ayrılır, örneğin eleman 1000 numaralı adrese yerleşir. Artık `head` `NULL` değil, 1000 göstermelidir. İçinde 9 olan bu elemanın `next` alanı `NULL`'dır.

Sonra yeni bir eleman gelir ve 1500 numaralı adrese yerleşir. Hoca sordu: Özel bir durum yoksa bu elemanı nereye yerleştirmek verimlidir? Önce "sona" ve "hemen arkasına" gibi yanıtlar geldi.

Soru (hocadan): Bu yeni gelen elemanı nereye yerleştireyim?

Cevap (öğrenciden): En başa koyup `head`'i ona eşitleyebiliriz. En sona kadar gidip oraya atamak yerine tek bir değişken değiştirmek daha kolay olmaz mı?

Hoca bu yanıtı onayladı. Yeni elemanı nereye ekleyeceğiniz ihtiyacınıza ve algoritmanızın tasarımına bağlıdır. Üç yol vardır:

1. **Başa ekleme.** Aksi söylenmediyse ve özel bir durum yoksa en verimli yol budur. Tek bir ekleme için boşuna n adım ilerleyip sona gitmek zorunda kalmazsınız. Bu örnekte `head` artık 1000'i değil 1500'ü gösterir, 1500'deki eleman da 1000'i gösterir.
2. **Sona ekleme.** Algoritmanız bunu gerektiriyorsa yapılır. Hocanın örneği: Daha önce çok kullanılanlar öndedir. Yeni gelen eleman ilk defa kullanılıyordur, bu yüzden en sonda durması istenir.
3. **Araya ekleme.** Yine algoritmanın ihtiyacına göre yapılır. Örneğin elemanlar sıralı duruyordur: 1000 numaralı adreste 2, sonra 4 vardır ve 6 gelmiştir. O zaman 6'yı sırayı bozmayacak yere yerleştirmelisiniz.

Hoca üçüncü duruma ikili arama ağacını yeniden örnek gösterdi. İkili arama ağacında da yeni elemanı istediğiniz yere yerleştiremezsiniz. 7 geldiyse karşılaştırırsınız: 5'ten büyük mü? Evet. 9'dan küçük mü? Evet. Bu sonuca göre 7'yi uygun yere yerleştirirsiniz. Bağlı listede de böyle ortada bir yere yerleştirme gerekir.

7. Başa Ekleme

Fonksiyon iki parametre alır: yer ayrılmış düğümün göstericisi ve eklenecek değer.

```
void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}
```

7.1 Listede Eleman Varsa

Verilen durum: `head` 1000'i gösteriyor. 1000'deki düğümde 9 var, bu düğüm 1200'ü gösteriyor ve liste böyle devam ediyor. Yeni düğüm 1500'de, eklenecek değer 8.

Adım	İşlem	Sonuç
1	`newNode->value = value`	1500'deki düğümün değeri 8 olur
2	`newNode->next = head`	1500'deki düğüm 1000'i gösterir
3	`head = newNode`	`head` 1500'ü gösterir

Sonuç olarak `head` ile eski ilk düğüm arasındaki bağ koparılmış olur. Liste artık `head` → 1500 (8) → 1000 (9) → 1200 → ... biçimindedir.

7.2 Liste Boşsa

Hoca hemen ikinci durumu kontrol etti: Eleman olmaması, `head`'in `NULL` olması demektir. Gelen eleman 1500 numaralı adreste, değeri 8.

Adım	İşlem	Sonuç
1	`newNode->value = value`	1500'deki düğümün değeri 8 olur
2	`newNode->next = head`	`head` `NULL` olduğu için `next` `NULL` olur
3	`head = newNode`	`head` 1500'ü gösterir

`head` ilk defa eklenen bu elemanı gösterir ve `next` alanı doğru biçimde `NULL` olur. Demek ki algoritma bu hâliyle boş liste için de doğru çalışır, ayrı bir kontrol gerektirmez.

8. Sona Ekleme

Verilen durum: `head` 1000'i gösteriyor. 1000'de 8 var ve bu düğüm 1500'ü gösteriyor. 1500'de 9 var ve bu düğüm `NULL` gösteriyor. Yeni gelen eleman bu listenin sonuna eklenecek.

> **Hocanın vurgusu:** İki durum vardır, bunları hiç unutmayın: Eleman varsa ve eleman yoksa, yani `head` `NULL` ise.

Başa eklemede `head`'in `NULL` olup olmadığını kontrol etmeye gerek yoktu, yapı buna çok müsaitti. Burada öyle değildir. Kontrol etmeden sona eklemeye kalkarsanız `NULL`'ın arkasına eklemeye çalışırsınız ve hatalı bir şey olur. Bu yüzden önce listenin boş olup olmadığına bakılır.

```
void insertAtEnd(NODE *newNode, int value)
{
    NODE *tmp;
    if (head == NULL)
        head = newNode;
    else
    {
        tmp = head;
        while (tmp->next != NULL)
            tmp = tmp->next;
        tmp->next = newNode;
    }
}
```

> **Ders sonrası not:** Ekrandaki bu kodda `newNode->value = value;` satırı yok. Hoca sözlü anlatımda da bu atamadan söz etmedi. `value` parametresi hiç kullanılmadığı için yeni düğümün değeri atanmamış kalır. Bu sürüm `gcc -Wall` ile uyarı vermeden derlendi. Hocanın 8, 9 listesine 4 eklendiğinde, ayrılan belleğin sabit bir desenle doldurulduğu çalıştırmada son değer 4 yerine `1515870810` yazıldı. Fonksiyonun başına `newNode->value = value;` eklenmelidir. Bölüm 10'un sonundaki tam programda bu satır vardır.

8.1 `next` Alanını `createNode` İçinde `NULL` Yapmak

Sona eklenen düğümün `next` alanı `NULL` olmalıdır. Hoca bu noktada bir ekleme yaptı: Bu atamayı her algoritmada ayrı ayrı yapmamak için düğüm oluşturulurken bir kez yapmak daha şıktır. Algoritmaların içinde de yapılabilir, ama başlangıçta bir yerde bulunması karışıklığı önler. `createNode` şu hâli alır:

```
NODE *createNode() {
    NODE *p;
    p = (NODE *) malloc(sizeof(NODE));
    if (p == NULL) {
        printf("Allocation error\n");
```

```
        exit(0);
    }
    p->next = NULL;
    return p;
}
```

Örneğin 1500 için yer tutuldu. Bu düğümün `next` alanı `NULL` yapıldı. Bundan sonraki bütün ekleme algoritmaları bu varsayımla çalışır.

8.2 Liste Boşsa

Yeni düğüm için örneğin 1200'de yer tutulmuştur. `head` `NULL` olduğu için doğrudan `head = newNode` yapılır, yani `head` 1200 olur. Düğümün `next` alanı zaten `NULL`'dır. Konu çözülmüştür.

8.3 Listede Eleman Varsa

Sona eklemek için sona kadar gitmek gerekir. Sona kadar gitmek, `next` alanı `NULL`'dan farklı olduğu sürece ilerlemek demektir. İlerlemeyi sağlamak için `tmp` adında geçici bir gösterici tanımlanır ve `head` ile başlatılır.

> **Ek bilgi (derste anlatılmadı):** İlerlerken `head` yerine ayrı bir `tmp` kullanılmasının nedeni şudur: `head` değişkeni ilerletilseydi ilk düğümün adresi kaybolurdu. Hoca `head`'i bu listeye işe başlanacak tek nokta olarak tanımlamıştı. Geçilen düğümlere bu değişken üzerinden yeniden ulaşılamazdı.

Hocanın örneği: Liste `head` → 1000 (8) → 1500 (9) → `NULL`. Yeni düğüm 1200'de, değeri 4.

Adım	`tmp`	`tmp->next`	Karar
1	1000	1500	`NULL` değil, `tmp = tmp->next` ile ilerle
2	1500	`NULL`	Döngüden çık
Sonuç	1500	1200	`tmp->next = newNode` ile 1500'deki düğüm 1200'ü gösterir

Son durum: `head` → 1000 (8) → 1500 (9) → 1200 (4) → `NULL`. Yeni düğümün `next` alanı oluşturulurken zaten `NULL` yapıldığı için başka bir işlem gerekmez.

9. `createNode` Nereden Çağrılmalı?

Soru: `createNode`'u `main` içinde çalıştırmak yerine ekleme fonksiyonlarının içinde çağırsak daha az parametre kullanmış olmaz mıyız?

Cevap: Olur, bu doğrudur. `createNode`'u `insert` fonksiyonunun kendisi de çağırabilir. Tek gereken şey, eklemeden önce düğümün oluşturulmuş olmasıdır. İhtiyacınıza göre nerede isterseniz orada oluşturabilirsiniz. Yer kullanımı veya zaman açısından bir fark olmaz.

Hoca bunları özellikle ayrı yazdığını söyledi. Bütün bir program yazmıyor, belirli bir senaryo için özel bir algoritma tasarlamıyordu. Düğüm oluşturmanın ve eklemenin nasıl yapıldığını bağımsız olarak görmemizi istiyordu. Ayrıca `main` yalnız bağlı listeyle ilgili iş yapmıyor olabilir. `main` bambaşka işler yapıyordur, bağlı liste yalnız bir fonksiyonun içinde kullanılıyordur. Program yazarken bunları ihtiyacınıza göre şekillendirmelisiniz.

10. Listeyi Yazdırma

Yazdırırken `head`'den başlanır ve bir gösterici `NULL` olana kadar gidilir. Hoca çıktının biçimiyle ilgilenmediğini, değişkeni yazdırmanın yeterli olduğunu söyledi. Bir sonraki elemana geçmek için `ptr = ptr->next` yapılır.

```c
void printList()
{
    NODE *ptr;
    ptr = head;
    while (ptr != NULL)
    {
        printf("%d\n", ptr->value);
        ptr = ptr->next;
    }
}
```

Hocanın örneği: `head` → 1000 (5) → 1200 (9) → 1500 (7) → `NULL`.

Adım	`ptr`	Yazılan	`ptr = ptr->next` sonrası
1	1000	5	1200
2	1200	9	1500
3	1500	7	`NULL`
4	`NULL`	Döngüden çıkılır	

Böylece 1000, 1200 ve 1500 numaralı adreslerdeki 5, 9 ve 7 ekrana yazılmış olur.

> **Hocanın vurgusu:** Bağlı listeyi ikinci ve üçüncü sınıfta durmadan kullanacaksınız. Ağaçlarda, graflarda, kuyruklarda çok fazla kullanmanız gerekecek. O yüzden lütfen düzgünce öğrenin.
>
> **Ek bilgi (derste anlatılmadı):** Hoca bütün bir program yazmadı. Aşağıda dersteki fonksiyonlar tek programda birleştirilmiştir. Ders sonrası notlardaki düzeltmeler uygulanmıştır: `stdio.h` eklendi, `insertAtEnd` içine değer ataması eklendi, `exit(0)` yerine `exit(EXIT_FAILURE)` yazıldı. Değerler hocanın başa ve sona ekleme örneklerindendir. Program `gcc -Wall` ile uyarısız derlendi ve 8, 9, 4 çıktısını verdi.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} NODE;

NODE *head;

NODE *createNode()
{
    NODE *p;
    p = (NODE *) malloc(sizeof(NODE));
    if (p == NULL) {
        printf("Allocation error\n");
        exit(EXIT_FAILURE);
    }
    p->next = NULL;
    return p;
}

void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}
```

```c
void insertAtEnd(NODE *newNode, int value)
{
    NODE *tmp;
    newNode->value = value;
    if (head == NULL)
        head = newNode;
    else
    {
        tmp = head;
        while (tmp->next != NULL)
            tmp = tmp->next;
        tmp->next = newNode;
    }
}

void printList()
{
    NODE *ptr;
    ptr = head;
    while (ptr != NULL)
    {
        printf("%d\n", ptr->value);
        ptr = ptr->next;
    }
}

int main()
{
    NODE *p;
    head = NULL;
    p = createNode();
    insertAtFront(p, 9);
    p = createNode();
    insertAtFront(p, 8);
    p = createNode();
    insertAtEnd(p, 4);
    printList();
    return 0;
}
```

11. İkili Ağacı Düz Dizide Tutmanın Sorunu

Araya eklemeye geçmeden önce hoca, ağaç yapısında neden düz dizi kullanılmaması gerektiğini gösterdi. Ağaç şöyledir: Kök 9. Kökün solunda 7, sağında 12 var. 7'nin solunda 5 var.

Bu ağaç dizide tutulsaydı 9, 7, 12, 5 biçiminde bir dizimiz olurdu. Diziyi kullanırken i. gözün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$. göze yerleşir. Sonra yeni eleman olarak 3 gelir ve 5'in soluna eklenir. 3'ün yeri hesaplanır: 5, 3. gözdedir, sol çocuğu $2 \cdot 3 + 1 = 7$. göze yerleşir.

Hoca sayarken bir gözü atladığını fark edip saymayı baştan yaptı: 3. gözden sonra aslında var olmayan 4, 5 ve 6. gözler vardır, 3 ise 7. gözdedir. Yeni elemanı yerleştirmek için aradaki gözleri boş bırakmak zorundasınız.

Bir de 3'ün sağına yeni eleman gelsin. Hoca bunun adresini de hesapladı: $2 \cdot 7 + 2 = 16$. Yeni eleman 16. göze yerleşir, arası bomboş kalır.

```
indeks : 0  1   2  3  4  5  6  7  8 ... 15  16
değer  : 9  7  12  5  -  -  -  3  -  ...  -  4
```

Altı eleman için dizinin en az 17 gözü, yani 0'dan 16'ya kadar yer kullanılmış olur (ara adım eklendi).

> **Hocanın vurgusu:** Bu problemi görmüş olmanız önemli. Ağaç yapısında düz dizi kullanmanın en büyük sıkıntısı budur.

Özellikle ikili ağaçta (binary tree), düz dizi kullanmanın sıkıntı yarattığı çok net görülür.

12. Araya Ekleme: Sıralı Liste

Hoca araya eklemenin mantığını anlattı, ama kodunu yazmadı. Öğrencilerden bilgisayar başında oturup sakin sakin yazmalarını istedi. Hocaya göre bir şeyi yapmadan yalnız dinleyerek iyice öğrenmek çok mümkün değildir.

Yine dikkat edilecek iki durum vardır.

12.1 Liste Boşsa

`head` `NULL` ise gelen eleman, örneğin 1200 numaralı adrese yerleşmiş eleman, doğrudan `head`'e bağlanır.

12.2 Listede Eleman Varsa

Hoca problemi şöyle sınırladı: Elemanlar bağlı listede küçükten büyüğe sıralı dursun. Bu arada araya eklemenin tek senaryosu bu değildir, her türlü problem olabilir.

Örnek: `head` → 1200 (3) → 1500 (4) → 1700 (9) → `NULL`. Yeni düğüm 1800'de, değeri 7.

Adım	Bulunulan düğüm	Karşılaştırma	Karar	Tutulan önceki adres (ara adım eklendi)
1	1200 (3)	7 > 3	1500'e ilerle	Henüz yok
2	1500 (4)	7 > 4	1700'e ilerle	1200
3	1700 (9)	7 < 9	9'dan önce gelmeli	1500

Araya yerleştirmek demek, 4'ün artık 1700'ü değil 1800'ü göstermesi, 7'nin ise 1700'ü göstermesi demektir. Son durum: `head` → 1200 (3) → 1500 (4) → 1800 (7) → 1700 (9) → `NULL`.

Soru (hocadan): 4'ün 7'yi göstermesini sağlamak için neyi bilmeniz lazım?

Cevap (öğrenciden): Hem 4'ü hem 9'u bilmeliyiz.

Hoca bunu düzelterek şunu söyledi: Yalnız `next` adreslerini bilmek yetmez. Nereden geldiğimizi de bilmemiz lazımdır, yani önceki (previous) adresi de bilmemiz gerekir.

> **Hocanın vurgusu:** Burada anahtar nokta budur: Araya eleman yerleştirebilmek için bir önce geldiğiniz adresi de bir değişkende tutmanız lazım. Bunu her ilerlemede yapmalısınız ki yerleştireceğiniz zaman kullanabilesiniz.
>
> **Ders sonrası not:** Hoca burada elde tutulması gereken adresi "1700" olarak söyledi. Doğrusu 1500'dür. 9'daki düğüme geldiğimizde 1700 zaten bulunulan adrestir ve 7'nin `next` alanına yazılır. Değiştirilecek olan 4'ün `next` alanıdır, buna da 4'ün düğümünün adresi olan 1500 ile ulaşılır. Hocanın aynı anda söylediği "bir önceki adresi tutun" kuralı da bunu gösterir.

Soru: Listenin sonuna gelip gelmediğimizi de, yani `next` alanının `NULL` olup olmadığını da kontrol etmemiz gerekiyor, değil mi? Sona geldiysek ve hâlâ yerini bulamadıysak ne olacak?

Cevap: Tabii. Örneğin 9'un arkasına 15 gelseydi, listenin bittiğini anladığınız noktada 15'i en sona yerleştirmeniz gerekir.

12.3 Hocanın Verdiği Alıştırma

Elemanların küçükten büyüğe sıralı olarak bulunduğu bir bağlı liste oluşturun. Verilen bir elemanı sırayı bozmayacak şekilde bu listeye yerleştirin.

Hoca listenin iki türlü oluşturulabileceğini de söyledi. Önce listeyi bir kerede oluşturabilirsiniz. Ya da her gelen elemanı döngü içinde sırayı bozmadan yerleştirirsiniz: 3'ü okuyup yerleştirirsiniz, 9'u yerleştirirsiniz, 5 gelince 5'i yerleştirirsiniz.

> **Ek bilgi (derste anlatılmadı):** Hoca bu kodu öğrencilerin yazmasını istedi, önce kendiniz deneyin. Aşağıdaki çözüm kısa nottaki örnektir, hoca yazmadı. Boş liste ve en küçük eleman durumunda `prev` `NULL` kalır ve `head` güncellenir. Dersteki sayılarla `gcc -Wall` ile derlenip çalıştırıldı: 3, 9, 5 sırayla eklenince liste 3, 5, 9 oldu. 3, 4, 9 listesine 7 ve 15 eklenince liste 3, 4, 7, 9, 15 oldu.

```
void insertSorted(NODE *newNode, int value)
{
    NODE *prev = NULL, *cur = head;
    newNode->value = value;
    while (cur != NULL && cur->value < value) {
        prev = cur;
        cur = cur->next;
    }
    newNode->next = cur;
    if (prev == NULL)
        head = newNode;
    else
        prev->next = newNode;
}
```

13. Sıradaki Konu: Düğüm Silme

Hoca son olarak tek bağlı listede bir düğümün nasıl silineceğine bakmak gerektiğini söyleyerek bu konuya geçti.

Dersin Toparlaması

Bu derste dizilerin sınırından yola çıkıldı. Statik dizide üst sınır baştan bilinmeli, dinamik dizide ise ardışık bir yer gerekir. Eleman sayısı bilinmediğinde ve ekleme ile silme sık yapıldığında her eleman için ayrı yer ayıran bağlı liste devreye girer. Düğüm, bir değer ile bir sonraki düğümün adresini tutar. `head` ilk düğümü gösterir, son düğümün `next` alanı `NULL`'dır. Bu yapının bedeli rastgele erişimin olmaması, gösterici için ek yer gerekmesi ve önbellek dostu olmamasıdır.

Tek bağlı listede düğüm oluşturma, başa ekleme, sona ekleme ve yazdırma kodlarıyla görüldü, araya ekleme ise mantığıyla anlatılıp alıştırma olarak bırakıldı. Her eklemede liste boşken ve doluyken ne olacağı ayrı düşünülür. Başa ekleme iki durumda da aynı kodla çalışır, sona ekleme boş listeyi kontrol etmek zorundadır, araya ekleme ise bir önceki düğümün adresini elde tutmayı gerektirir. Sıradaki konu tek bağlı listeden düğüm silmedir, ardından çift bağlı liste gelecektir. Bu yapı ileride ağaçlarda, graflarda ve kuyruklarda sürekli kullanılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.